

```
>>> if 'cats' in numberOfPets: # Check if 'cats' exists as a key.
...     print('I have', numberOfPets['cats'], 'cats.')
... else:
...     print('I have 0 cats.')
...
I have 0 cats.
```

This code checks whether the string 'cats' exists as a key in the `numberOfPets` dictionary. If it does, a `print()` call accesses `numberOfPets['cats']` as part of a message for the user. If it doesn't, another `print()` call prints a string without accessing `numberOfPets['cats']` so it doesn't raise a `KeyError`.

This pattern happens so often that dictionaries have a `get()` method that allows you to specify a default value to return when a key doesn't exist in the dictionary. The following pythonic code is equivalent to the previous example:

```
>>> # Pythonic Example
>>> numberOfPets = {'dogs': 2}
>>> print('I have', numberOfPets.get('cats', 0), 'cats.')
I have 0 cats.
```

The `numberOfPets.get('cats', 0)` call checks whether the key 'cats' exists in the `numberOfPets` dictionary. If it does, the method call returns the value for the 'cats' key. If it doesn't, it returns the second argument, 0, instead. Using the `get()` method to specify a default value to use for nonexistent keys is shorter and more readable than using `if-else` statements.

Conversely, you might want to set a default value if a key doesn't exist. For example, if the dictionary in `numberOfPets` doesn't have a 'cats' key, the instruction `numberOfPets['cats'] += 10` would result in a `KeyError` error. You might want to add code that checks for the key's absence and sets a default value:

```
>>> # Unpythonic Example
>>> numberOfPets = {'dogs': 2}
>>> if 'cats' not in numberOfPets:
...     numberOfPets['cats'] = 0
...
>>> numberOfPets['cats'] += 10
```